Lab 3: Sheep & Wolves

Scot Schmidl
sschmidl3@gatech.edu

1 HOW DOES YOUR AGENT WORK?

In this project, I implemented a search-based agent that treats Sheep & Wolves as a graph over states (S, W, B), where S is the number of sheep on the left bank, W is the number of wolves on the left bank, and B is the boat's position (0 for left, 1 for right). The initial state is the number of sheep, the number of wolves, and the boat on the left side, and the goal is no sheep or wolves on the left side and the boat on the right side. I pre-defined the legal boat moves as the five possible moves, i.e. (1, 0), (2, 0), (0, 1), (0, 2), (1, 1), where the left is how many sheep and the right is how many wolves.

The core algorithm used is breadth-first search (BFS) over the state space. I maintain a queue, `to_visit`, which stores the current state and the path of moves used to reach that state. I also track `visited` as a set of states to avoid revisiting the same combination of animals and boat position, which helps guarantee termination. At each step, the agent removes the next state from the queue, checks whether it matches the goal, and if so returns the path immediately. If not, it generates new states by iterating over the legal moves and applying them either left to right or right to left depending on the boat's position.

State generation is done by simple arithmetic. When the boat is on the left, the agent subtracts the number of sheep and wolves from the left side, and flips the boat to the right side. When the boat is on the right, it adds the number of sheep and wolves back to the left side, and flips the boat back to the left side. In each case, it computes the right side totals as `initial_sheep - new_left_sheep` and `initial_wolves - new_left_wolves` so the total number is maintained.

After generation of the next state, the agent enforces the constraints to ensure the state is valid. It will reject a state if the left or right sides go negative, then it will ensure the wolves never outnumber the sheep, but if there are no sheep then any number of wolves are allowed. Only the states that pass these checks and have not been visited before are added to the queue with the path of moves.

Since I used BFS and never revisit states, the first time the agent reaches the goal,

the current path is guaranteed to be the shortest path by number of moves. If the queue ever becomes empty without finding the goal, the agent determines that there is no valid path and returns an empty list.

2 HOW WELL DOES YOUR AGENT PERFORM?

My BFS agent performs well locally and in Gradescope solving 7/7 and 40/40, respectively, meaning it finds correct and optimal solutions. It returns moves that safely move all animals and never violate the constraints. The agent performs well on the 25 random cases, as well. For solvable cases, it always finds a solution because every legal state that can be reached from the initial states is eventually generated and tested. For unsolvable cases, the queue exhausts all reachable states without ever reaching the goal, and returns an empty list. I did not observe any particular pattern of cases that were too hard for my agent to solve. The runtime increases as the number of moves and animals increases, but there are no cases where the agent fails.

3 HOW EFFICIENT IS YOUR AGENT?

From a complexity perspective, the cost of the algorithm is controlled by the size of the state space and doing a BFS over that space. Each state is (S, W, B), where S and W range from 0 to the initial sheep and wolves count, and B is either 0 or 1. In the worst case there are (initial sheep + 1) $\times$ (initial wolves + 1) $\times$ 2 possible states. The overall time complexity becomes $O(N \cdot M)$, where N and M are the number of sheep and wolves, respectively. Due to the `visited` set, the runtime scales linearly with the number of legal states. As the number of animals increases, the depth will increase, but many states will be pruned by the constraints. Space complexity is also $O(N \cdot M)$ to store the `visited` set and the queue.

The agent runs quickly on all cases it saw, whether a small or large number of animals, the cases are solved almost instantly. The fixed move set and constraint checks mean that many illegal states are removed before they ever reach the queue, which helps keep both runtime and memory manageable.

4 DOES YOUR AGENT DO ANYTHING CLEVER TO ARRIVE AT AN ANSWER MORE EFFICIENTLY?

My agent is implementing a standard state-space search with explicit states, operators, and constraints, which mirrors the planning and search topics from the lectures. The most clever aspect of my agent is that I encoded the domain constraints so that the search space is pruned while still having a guarantee of completeness. By enforcing the wolves-greater-than-sheep rule and discarding any state where the counts go negative on both banks, the agent is able to avoid exploring large parts of the space of possible moves that could never be a part of a valid solution.

Breadth-first search was a design choice because it's simple to implement, can get to the shortest solution by number of trips, and doesn't require designing a specific heuristic, which would be more work for the relatively small problem sizes. The use of a fixed set of moves, `self.legal_moves`, encodes domain knowledge, i.e. we never have to consider moving more than two animals or leaving the boat empty, so the agent doesn't waste time evaluating illegal or irrelevant moves. Overall, I would describe the agent as clever enough for this particular case.

5 HOW DOES YOUR AGENT COMPARE TO A HUMAN?

To solve a problem like Sheep & Wolves, or Guards & Prisoners (from the lecture), I would solve this in a similar way but less systematic. I would start by writing down the same set of legal moves, because having them visually in front of me makes it easier to think about combinations and remember constraints. From there, I would mentally keep track of the constraint that wolves can't outnumber sheep and try moves that appear safe. I would send small groups across, bring one animal back if I get stuck, and look for patterns that make sense.

My agent, by contrast, does a systematic enumeration that I might avoid, i.e. it represents each combination as a state, (S, W, B), generates all legal states by applying each move template to the state, and then tests each state against the constraints that I would track in my head. Instead of relying on intuition or pattern recognition, my agent runs the BFS, exploring every valid combination reachable from the initial state until it reaches the goal or doesn't find a solution. So while both my agent and I reason with the same basic ideas, the agent's strategy is exhaustive and will find the optimal solution, while my strategy is

more heuristic, i.e. I would follow a small number of patterns and backtrack by hand when I get stuck. I might be more likely to make a mistake on larger cases than the agent would.

Overall, I would say that my agent is closer to how I would like to solve the puzzle rather than how I would actually do it. It formalizes the same constraints and legal moves I think about, but then applies them more systematically in a way that I might struggle by hand, especially for large numbers of animals.